1. Weak vs Strong Learning

1. Definitions

Weak learner: Produces classifier with weighted error $\leq \frac{1}{2} - \gamma$ for some $\gamma > 0$.

Strong learner: Produces classifier with arbitrarily small error given enough samples.

Boosting question: Is weak learning equivalent to strong learning?

Answer: Yes! Boosting algorithms convert weak learners into strong learners.

2. Weak Learning Assumption

For any distribution D over training examples, weak learner outputs h with:

$$\varepsilon_D(h) \leq \frac{1}{2} - \gamma$$

where $\varepsilon_D(h) = \sum_{i=1}^{m} D(i) \cdot \mathbb{1}(h(x_i) \neq y_i)$ is the **weighted error** and $\gamma > 0$ is the **edge** (advantage over random guessing).

Key insight: Even slight improvement over random (49% vs 50%) is enough!

2. AdaBoost Algorithm

1. Core Idea

Strategy: Iteratively train weak learners, each focusing on examples that previous learners got wrong.

Mechanism: Maintain distribution D_t over training examples. Increase weight on misclassified examples, decrease on correctly classified ones.

Output: Weighted majority vote of all weak learners.

2. Algorithm (T rounds)

Initialize: $D_1(i) = \frac{1}{m}$ for all i

For $t = 1, ..., T$:

1. Train weak learner on distribution D_t, get h_t
2. Compute weighted error:

$$\varepsilon_t = \sum_{i=1}^{m} D_t(i) \cdot \mathbb{1}(h_t(x_i) \neq y_i)$$

3. Compute classifier weight:

$$\alpha_t = \frac{1}{2} \ln \frac{1 - \varepsilon_t}{\varepsilon_t}$$

4. Update distribution:

$$D_{t+1}(i) = \frac{D_t(i) \exp(-\alpha_t y_i h_t(x_i))}{Z_t}$$

where Z_t is normalization constant.

Output: $H(x) = \text{sign}\left(\sum_{t=1}^{T} \alpha_t h_t(x)\right)$

3. Weight Update Intuition

Correct prediction ($y_i h_t(x_i) = +1$):

- $\exp(-\alpha_t \cdot 1) < 1 \rightarrow$ weight decreases

Incorrect prediction ($y_i h_t(x_i) = -1$):

- $\exp(-\alpha_t \cdot (-1)) = \exp(\alpha_t) > 1 \rightarrow$ weight increases

Magnitude: α_t larger when ε_t smaller (more confident classifier gets more weight).

3. Exponential Loss Minimization

1. Loss Function

AdaBoost minimizes **exponential loss**:

$$L(F) = \sum_{i=1}^{m} \exp(-y_i F(x_i))$$

where $F(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$ is the combined classifier.

Why exponential? Smooth, convex, differentiable (unlike 0-1 loss).

2. Coordinate Descent View

Key insight: AdaBoost performs **coordinate descent** on exponential loss in function space.

At round t, fix $F_{t-1} = \sum_{s=1}^{t-1} \alpha_s h_s$ and optimize:

$$\min_{\alpha, h} \sum_{i=1}^{m} \exp(-y_i(F_{t-1}(x_i) + \alpha h(x_i)))$$

Greedy step:

1. Choose h_t to minimize weighted error under D_t
2. Choose α_t optimally given h_t
3. Update $F_t = F_{t-1} + \alpha_t h_t$

3. Training Error Bound

Theorem: After T rounds, training error is bounded by:

$$\text{TrainError} \leq \exp(-2\gamma^2 T)$$

where γ is the weak learning edge.

Implication: Training error decreases **exponentially fast** with number of rounds!

Proof sketch: Show $Z_t \leq \sqrt{1 - 4\gamma^2}$ and use $\text{TrainError} \leq \prod_{t=1}^{T} Z_t$.

4. Convexity and Convergence

1. Why Exponential Convergence?

Key property: Exponential loss is **convex**.

Gradient: $\nabla L = \sum_{i=1}^{m} -y_i \exp(-y_i F(x_i)) \cdot \nabla F(x_i)$

Connection to D_t: Distribution $D_t(i) \propto \exp(-y_i F_{t-1}(x_i))$ is proportional to gradient!

Result: Greedy coordinate descent makes consistent progress, yielding exponential convergence.

2. Loss vs Error

Exponential loss: Upper bounds 0-1 loss

$$\mathbb{1}(yF(x) \leq 0) \leq \exp(-yF(x))$$

Training error: Number of misclassifications

$$\text{TrainError} = \frac{1}{m} \sum_{i=1}^{m} \mathbb{1}(y_i F(x_i) \leq 0)$$

Bound: $\text{TrainError} \leq \frac{1}{m} \sum_i \exp(-y_i F(x_i)) = \frac{L(F)}{m}$

5. Practical Considerations

1. Weak Learner Choices

Decision stumps: Single-split decision trees (most common)

Requirements:

- Polynomial time training
- Edge $\gamma > 0$ on any distribution
- No need to be highly accurate

Examples: Stumps, linear classifiers, small trees

2. Hyperparameters

Number of rounds T:

- Trade-off: More rounds → lower training error, potential overfitting
- In practice: Cross-validate or use early stopping
- Theory: Can run indefinitely (margin explanation, next section)

Base learner complexity:

- Simpler base learners (e.g., depth-1 trees) often work best
- Prevents individual learners from overfitting

3. Implementation Tips

1. **Feature scaling:** Not critical (unlike gradient descent)
2. **Initialization:** Uniform distribution D_1
3. **Edge requirement:** Ensure base learner achieves $\varepsilon_t < \frac{1}{2}$
4. **Numerical stability:** Clip α_t to avoid extreme weights
5. **Early stopping:** Monitor validation error

6. Connection to Other Methods

1. Comparison to Bagging

Bagging (Bootstrap Aggregating):

- Sample with replacement, train independently
- Equal weight to all classifiers
- Reduces variance through averaging

Boosting:

- Reweight examples adaptively
- Weight classifiers by performance
- Reduces bias by focusing on hard examples

2. Gradient Boosting

Generalization: Replace exponential loss with arbitrary differentiable loss $L(y, F(x))$.

Algorithm: Fit weak learners to **negative gradient** of loss:

$$h_t \approx -\nabla_F L(y, F_{t-1}(x))$$

Examples:

- Squared loss → L2Boosting
- Logistic loss → LogitBoost
- Arbitrary loss → Gradient Boosting Machines (GBM)

7. Key Pitfalls

Boosting $\neq$ averaging: AdaBoost uses **weighted** voting, not simple averaging. Classifier weights α_t depend on performance.

Weak learning edge: If $\varepsilon_t \geq \frac{1}{2}$ (worse than random), algorithm fails. Need $\gamma > 0$.

Training error reaching zero: Unlike typical models, AdaBoost can continue improving **after** training error hits zero (explained by margin theory).

Coordinate descent = greedy: AdaBoost finds h_t and α_t greedily, not globally optimal. But convexity ensures consistent progress.

Distribution vs sampling: D_t is a probability distribution over examples, not a random sample. Used to define weighted error for weak learner.